
SQL Practice

Vishwas Singh,SME

Contents

Northwind Database Practice with Solutions	3
1. Basic Selection and Sorting	3
2. Filtering and Logistics	4
3. Advanced Filtering and Calculations	4
4. Joins and Aggregations	5
5. Views and DDL	6
6. String and Date Functions	6
7. Views	7
8. Procedures	8

VKS

Northwind Database Practice with Solutions

This document contains the SQL solutions for the 50 practice queries based on the Northwind database.

1. Basic Selection and Sorting

- **Category Report:** Shows CategoryName and Description from the categories table sorted by name.

```
SELECT CategoryName, Description
FROM categories
ORDER BY CategoryName;
```

- **Customer Contact Info:** Shows contact details from the customers table sorted by phone number.

```
SELECT ContactName, CompanyName, ContactTitle, Phone
FROM customers
ORDER BY Phone;
```

- **Capitalized Employee Names:** Displays FirstName and LastName in uppercase, renamed accordingly, and sorted by HireDate from newest to oldest.

```
SELECT UPPER(FirstName) AS FirstName, UPPER(LastName) AS Lastname, HireDate
FROM employees
ORDER BY HireDate DESC;
```

- **Top 10 Freight Orders:** Shows the top 10 orders based on freight cost in descending order.

```
SELECT TOP 10 OrderID, OrderDate, ShippedDate, CustomerID, Freight
FROM orders
ORDER BY Freight DESC;
```

- **Customer ID Lowercase:** Renames CustomerID to ID and converts it to lowercase.

```
SELECT LOWER(CustomerID) AS ID
FROM customers;
```

2. Filtering and Logistics

- **Supplier Directory:** Shows supplier info sorted by Country (descending) then CompanyName (ascending).

```
SELECT CompanyName, Fax, Phone, Country, HomePage
FROM suppliers
ORDER BY Country DESC, CompanyName ASC;
```

- **Buenos Aires Customers:** Filters customers specifically from 'Buenos Aires'.

```
SELECT CompanyName, ContactName
FROM customers
WHERE City = 'Buenos Aires';
```

- **Out of Stock Products:** Lists products where units in stock are zero.

```
SELECT ProductName, UnitPrice, QuantityPerUnit
FROM products
WHERE UnitsInStock = 0;
```

- **International Customers (Exclusion):** Customers not from Germany, Mexico, or Spain.

```
SELECT ContactName, Address, City
FROM customers
WHERE Country NOT IN ('Germany', 'Mexico', 'Spain');
```

- **Specific Date Orders:** Orders placed on May 21, 1996.

```
SELECT OrderDate, ShippedDate, CustomerID, Freight
FROM orders
WHERE OrderDate = '1996-05-21';
```

3. Advanced Filtering and Calculations

- **International Employees:** Employees not from the United States

```
SELECT FirstName, LastName, Country
FROM employees
WHERE Country <> 'UK'; -- Note: Northwind typically uses 'UK' and 'USA'
```

- **Late Shipments:** Orders where the shipped date exceeds the required date.

```
SELECT EmployeeID, OrderID, CustomerID, RequiredDate, ShippedDate
FROM orders
WHERE ShippedDate > RequiredDate;
```

- **City Wildcards:** Customers from cities starting with A or B.

```
SELECT City, CompanyName, ContactName
FROM customers
WHERE City LIKE 'A%' OR City LIKE 'B%';
```

- **Even and Odd Order IDs:**

- **Even:** SELECT OrderID FROM orders WHERE OrderID % 2 = 0;
- **Odd:** SELECT OrderID FROM orders WHERE OrderID % 2 <> 0;

- **Products for Reorder:** Products that have reached their reorder level.

```
SELECT ProductName, UnitsInStock, UnitsOnOrder, ReorderLevel
FROM products
WHERE UnitsInStock + UnitsOnOrder <= ReorderLevel;
```

- **Missing Fax Numbers:** Customers without a fax number, sorted by ContactName.

```
SELECT CompanyName, ContactName, Fax
FROM customers
WHERE Fax IS NULL
ORDER BY ContactName;
```

4. Joins and Aggregations

- **Order Detail Totals:** Grouped by OrderID and sorted by the number of items.

```
SELECT OrderID, COUNT(OrderID) AS NumberofOrders
FROM [order details]
GROUP BY OrderID
ORDER BY NumberofOrders DESC;
```

- **Three-Table Join:** Joining Products, Suppliers, and Categories.

```
SELECT s.SupplierID, s.CompanyName, c.CategoryName, p.ProductName, p.UnitPrice
FROM products p
JOIN suppliers s ON p.SupplierID = s.SupplierID
JOIN categories c ON p.CategoryID = c.CategoryID;
```

- **Self-Join (Employee/Manager):** Shows who employees report to.

```
SELECT e.EmployeeID,  
       e.FirstName + ' ' + e.LastName AS employee,  
       m.FirstName + ' ' + m.LastName AS manager  
FROM employees e  
LEFT JOIN employees m ON e.ReportsTo = m.EmployeeID  
ORDER BY e.EmployeeID;
```

- **High Freight Aggregation:** Grouped by CustomerID with sum of freight > \$200.

```
SELECT CustomerID, SUM(Freight)  
FROM orders  
GROUP BY CustomerID  
HAVING SUM(Freight) > 200;
```

5. Views and DDL

- **Create/Manage Views:** Creating and dropping the CustomerInfo and Product Details views.

```
CREATE VIEW CustomerInfo AS  
SELECT c.CustomerID, c.CompanyName, o.OrderDate, o.ShippedDate  
FROM customers c JOIN orders o ON c.CustomerID = o.CustomerID;  
  
-- Change name  
EXEC sp_rename 'CustomerInfo', 'customer details';  
  
-- Drop view  
DROP VIEW [customer details];
```

- **Table Duplication:** Creating a copy of the shippers table.

```
SELECT * INTO shippers_duplicate FROM shippers;  
  
-- Drop table  
DROP TABLE shippers_duplicate;
```

6. String and Date Functions

- **Contact Info Concatenation:** Formats employee reachability info.

```
SELECT FirstName + ' ' + LastName + ' can be reached at x' + Extension AS  
    ↳ ContactInfo  
FROM employees;
```

- **Employee Age calculation:** Calculating age based on birthdate.

```
SELECT LastName, FirstName, Title,  
    CONVERT(VARCHAR, DATEDIFF(YEAR, BirthDate, GETDATE())) + ' Years' AS  
    ↳ Age  
FROM employees;
```

- **Product Description Formatting:** Concatenating weight and cost info

```
SELECT ProductName + ' weighs/is ' + QuantityPerUnit + ' and cost $' +  
    ↳ CAST(CAST(UnitPrice AS INT) AS VARCHAR) AS ProductInfo  
FROM products;
```

7. Views

- **Category Sales Performance** This view calculates the total revenue for each product category.

```
CREATE VIEW CategoryRevenue AS  
SELECT  
    c.CategoryName,  
    SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS TotalRevenue  
FROM Categories c  
JOIN Products p ON c.CategoryID = p.CategoryID  
JOIN [Order Details] od ON p.ProductID = od.ProductID  
GROUP BY c.CategoryName;
```

- **Active Customer List** A view to quickly identify customers who have placed an order within a specific period.

```
CREATE VIEW ActiveCustomers AS  
SELECT DISTINCT  
    c.CompanyName,  
    c.ContactName,  
    c.Phone  
FROM Customers c  
JOIN Orders o ON c.CustomerID = o.CustomerID  
WHERE o.OrderDate > '1997-01-01';
```

- **Employee Territory Mapping** This query uses a Triple Join to link employees to their specific regions.

```
SELECT
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    t.TerritoryDescription,
    r.RegionDescription
FROM Employees e
JOIN EmployeeTerritories et ON e.EmployeeID = et.EmployeeID
JOIN Territories t ON et.TerritoryID = t.TerritoryID
JOIN Region r ON t.RegionID = r.RegionID;
```

- **Full Product-Supplier-Category Breakdown** A comprehensive report showing which suppliers provide which categories.

```
SELECT
    s.CompanyName AS Supplier,
    c.CategoryName,
    COUNT(p.ProductID) AS ProductCount
FROM Suppliers s
LEFT JOIN Products p ON s.SupplierID = p.SupplierID
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY s.CompanyName, c.CategoryName
ORDER BY Supplier;
```

8. Procedures

- **Get Customer Order History** This procedure takes a CustomerID as input and returns all products purchased by that customer.

```
CREATE PROCEDURE GetCustomerPurchaseHistory
    @CustID nchar(5)
AS
BEGIN
    SELECT
        p.ProductName,
        SUM(od.Quantity) AS TotalPurchased
    FROM Products p
    JOIN [Order Details] od ON p.ProductID = od.ProductID
    JOIN Orders o ON od.OrderID = o.OrderID
```



```
WHERE o.CustomerID = @CustID
GROUP BY p.ProductName;
END;
-- To run: EXEC GetCustomerPurchaseHistory @CustID = 'ALFKI'
```

- **Update Product Price by Category** A procedure to apply a percentage-based price increase to an entire category.

```
CREATE PROCEDURE UpdateCategoryPrice
    @CatName nvarchar(15),
    @Percentage decimal(5,2)
AS
BEGIN
    UPDATE Products
    SET UnitPrice = UnitPrice * (1 + @Percentage)
    FROM Products p
    JOIN Categories c ON p.CategoryID = c.CategoryID
    WHERE c.CategoryName = @CatName;
END;
-- To run: EXEC UpdateCategoryPrice @CatName = 'Seafood', @Percentage = 0.10
```